

ECO764 Assignment 1

Aayush Kumar
Roll No: 230027

Data Description

The analysis uses daily returns of the BSE Sensex index over the period 2000–2025. The figure below plots the daily percentage log returns computed from adjusted closing prices. The return series fluctuates around zero and exhibits periods of high and low volatility over time. Several large positive and negative return realizations are visible, particularly during major market stress periods, indicating substantial variability in daily stock market returns.

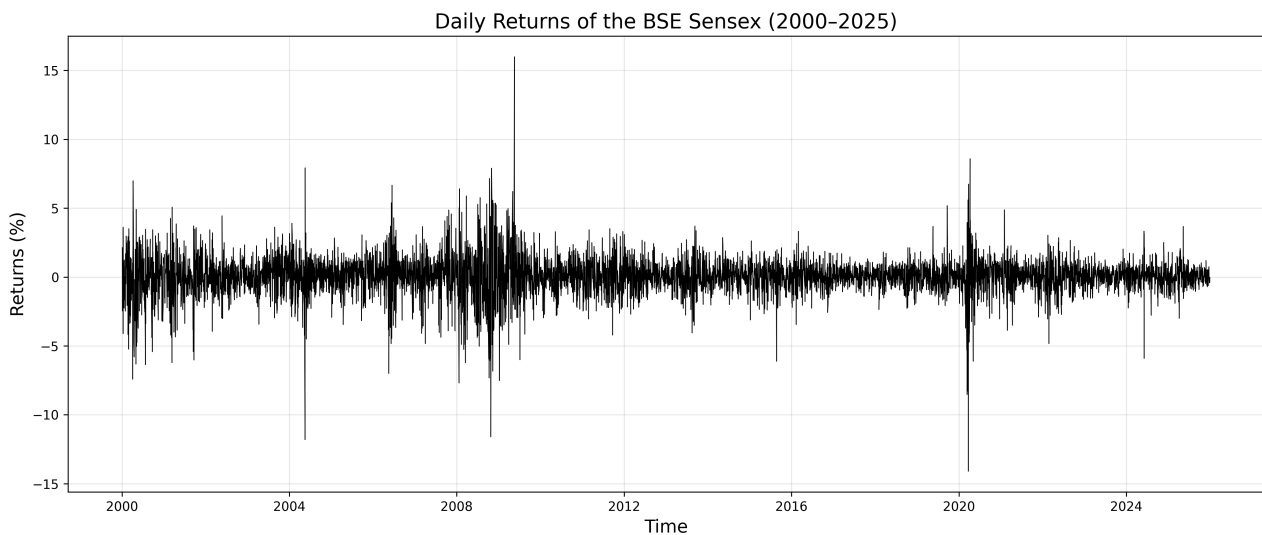


Figure 1: Daily Returns of the BSE Sensex (2000–2025)

Appendix: Python Code for Data Download and Return Computation

Listing 1: Python code for downloading BSE Sensex data and computing daily returns

```
1 import yfinance as yf
2 import pandas as pd
3 import numpy as np
4
5 # Download BSE Sensex data
6 start_date = "2000-01-01"
7 end_date = "2025-12-31"
8
9 data = yf.download("^BSESN", start=start_date, end=end_date)
10
11 # Use Close prices
12 prices = data["Close"]
13
14 # Compute daily log returns (in percentage)
15 returns = np.log(prices / prices.shift(1)) * 100
16 returns = returns.dropna()
17
18 # Basic check
19 returns.head()
```

Listing 2: Python code for generating and saving the daily returns plot

```

1 import matplotlib.pyplot as plt
2 import os
3
4 # Create the 'figures' directory if it does not exist
5 if not os.path.exists("figures"):
6     os.makedirs("figures")
7
8 # Plot daily returns
9 plt.figure(figsize=(14, 6))
10 plt.plot(returns, color="black", linewidth=0.6)
11 plt.title("Daily Returns of the BSE Sensex (2000--2025)", fontsize=16)
12 plt.xlabel("Time", fontsize=14)
13 plt.ylabel("Returns (%)", fontsize=14)
14 plt.grid(alpha=0.3)
15 plt.tight_layout()
16
17 # Save figure
18 plt.savefig("figures/returns_big.png", dpi=300)
19 plt.close()

```

Question 1: Small Average Returns

(a) Mean and Standard Deviation

Table 1 reports summary statistics of daily BSE Sensex returns, including the mean, standard deviation, and extreme values. The mean daily return is positive but very small in magnitude, while the standard deviation is substantially larger, indicating considerable day-to-day variability. The wide range between the minimum and maximum returns highlights the presence of large market movements over the sample period.

Table 1: Summary Statistics of Daily BSE Sensex Returns

Statistic	Value
Mean daily return(%)	0.04303
Standard deviation(%)	1.37979
Minimum daily return(%)	-14.10174
Maximum daily return(%)	15.98998
Mean / Volatility ratio	0.03118

(b) Mean Return Relative to Volatility

The mean daily return of the BSE Sensex is approximately 0.043%, while the standard deviation is about 1.380%. The ratio of the mean return to volatility is 0.031, indicating that the average return is extremely small relative to daily fluctuations. This implies that short-term movements in stock prices are dominated by volatility rather than by the mean return. Figure 1 illustrates the large day-to-day variability in returns around zero throughout the sample period.

(c) Difficulty of Estimating Expected Returns at High Frequency

At the daily frequency, expected stock returns are difficult to estimate precisely because volatility is much larger than the mean. Despite using a long sample period from 2000 to 2025, the mean daily return of 0.043% is small relative to the standard deviation of 1.380%, resulting in substantial sampling uncertainty. Consequently, estimates of expected returns at high frequency are imprecise and statistically noisy, which represents a key stylised fact of financial returns.

Appendix: Python Code for Question 1 Summary Statistics

Listing 3: Python code for computing summary statistics of daily BSE Sensex returns

```

1 # Summary statistics for Question 1
2
3 mean_return = returns.mean()
4 std_return = returns.std()
5 min_return = returns.min()
6 max_return = returns.max()
7 mean_vol_ratio = mean_return / std_return
8
9 # Print results (to be reported in LaTeX tables)
10 print("Summary Statistics of Daily BSE Sensex Returns")
11 print(f"Mean daily return (%)           : {mean_return.item():.5f}")
12 print(f"Standard deviation (%)          : {std_return.item():.5f}")
13 print(f"Minimum daily return (%)          : {min_return.item():.5f}")
14 print(f"Maximum daily return (%)           : {max_return.item():.5f}")
15 print(f"Mean / Volatility ratio             : {mean_vol_ratio.item():.5f}")

```

Question 2: Serial Correlation in Returns

(a) Sample Autocorrelation Function (ACF) of Returns

Figure 2 presents the sample autocorrelation function (ACF) of daily BSE Sensex returns. The autocorrelation coefficients at all lags are close to zero and lie within the approximate 95% confidence bands shown in the figure. These bands are based on large-sample normal approximations commonly used for ACF estimation. The results indicate the absence of significant linear dependence in daily returns.

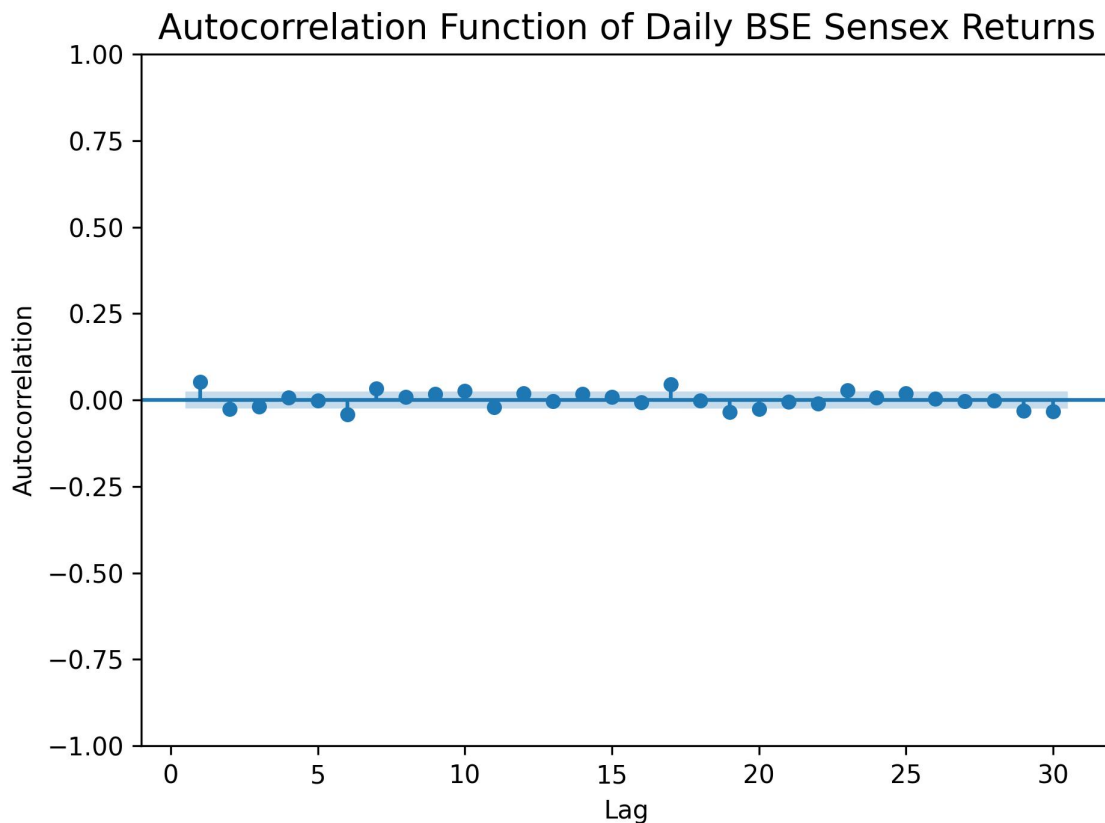


Figure 2: Sample Autocorrelation Function of Daily BSE Sensex Returns

(b) First-Order Autocorrelation

A formal test of first-order autocorrelation is conducted using the sample correlation between r_t and r_{t-1} . The estimated lag-one correlation is approximately 0.053 and is statistically significant at the 5% level. However, the magnitude of the correlation is very small, indicating that the statistical significance is driven by the large sample size rather than by economically meaningful dependence.

(c) Implications for Return Predictability

Although the first-order autocorrelation is statistically significant, its magnitude is economically negligible. Such a small correlation provides little scope for meaningful return predictability in practice. Overall, daily BSE Sensex returns exhibit behavior close to a white-noise process, which is consistent with weak-form market efficiency. Past returns therefore offer limited usefulness for forecasting future returns.

Appendix: Python Code for Question 2 (Serial Correlation)

Listing 4: Python code for plotting the ACF of daily BSE Sensex returns

```
1 import matplotlib.pyplot as plt
2 from statsmodels.graphics.tsaplots import plot_acf
3
4 # ACF plot of returns
5 plt.figure(figsize=(10, 5))
6 plot_acf(returns, lags=30, zero=False)
7 plt.title("Autocorrelation Function of Daily BSE Sensex Returns", fontsize=14)
8 plt.xlabel("Lag")
9 plt.ylabel("Autocorrelation")
10 plt.tight_layout()
11
12 # Save figure
13 plt.savefig("figures/q2_acf_returns.jpg", dpi=300)
14 plt.close()
```

Listing 5: Python code for testing first-order autocorrelation using Pearson correlation

```
1 import numpy as np
2 from scipy import stats
3
4 # Compute lag-1 autocorrelation
5 r_t = returns.values[1:]
6 r_t1 = returns.values[:-1]
7
8 corr, p_value = stats.pearsonr(r_t, r_t1)
9
10 corr, p_value
```

Question 3: Volatility Clustering

(a) Daily Returns and Volatility Patterns

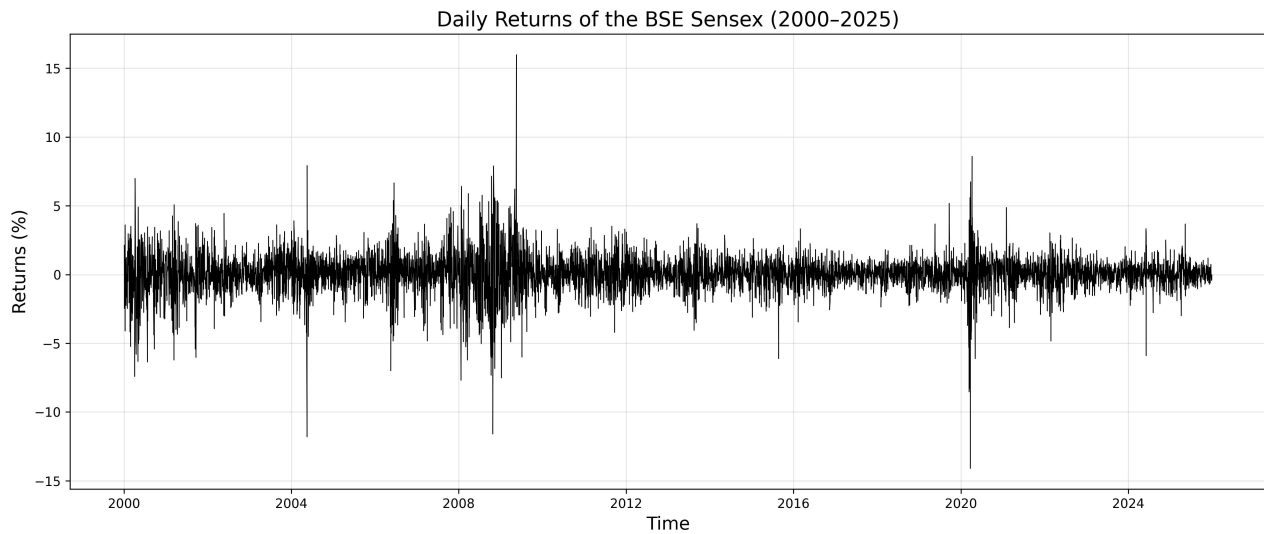


Figure 3: Daily Returns of the BSE Sensex (2000–2025)

Figure 3 shows that the volatility of daily BSE Sensex return varies substantially over time. Periods of high volatility are characterized by large and frequent return fluctuations and are clearly visible in the early 2000s, with notable spikes around 2004, during the global financial crisis of 2008–2009, and again during the COVID-19 shock in 2020. In contrast, periods of low volatility are observed in between these episodes, where returns exhibit smaller and more stable fluctuations, particularly during the mid-2000s and several years following 2012. This visual evidence suggests that market volatility alternates between turbulent and calm regimes rather than remaining constant over time.

(b) Autocorrelation of Squared Returns

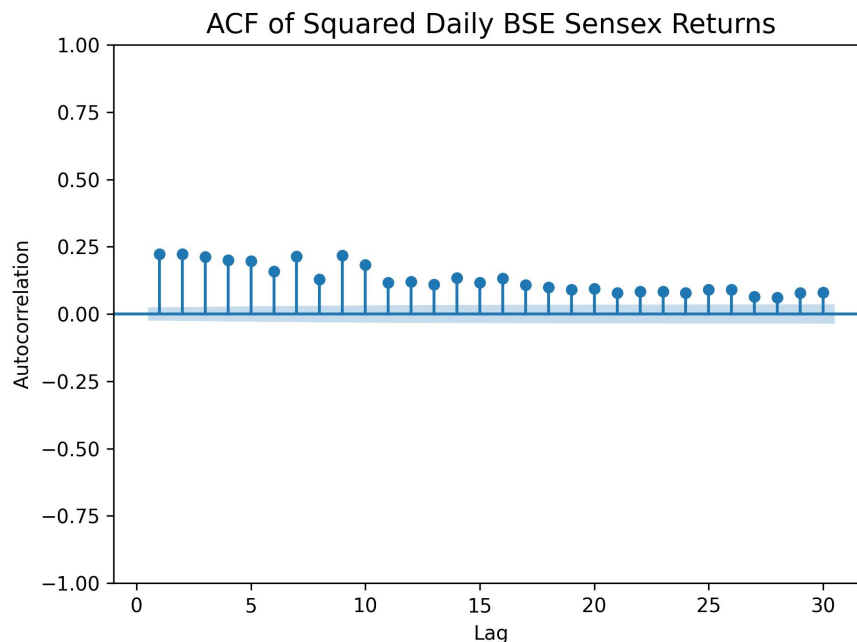


Figure 4: Autocorrelation Function of Squared Daily BSE Sensex Returns

Figure 4 displays the sample autocorrelation function of squared daily BSE Sensex returns. The autocorrelations are positive and statistically significant across many lags, with values remaining well above zero and lying outside

the approximate confidence bands for a prolonged range of lags. Moreover, the autocorrelation decays slowly as the lag increases, indicating strong persistence in return volatility.

(c) Evidence of Volatility Clustering

The results from parts (a) and (b) provide strong evidence of volatility clustering in daily BSE Sensex returns. The time-series plot of returns shows that periods of large price fluctuations tend to occur together, with high-volatility episodes followed by further high volatility and calm periods followed by relatively low volatility. In addition, the ACF of squared returns exhibits positive and persistent autocorrelation across many lags, indicating that volatility is serially correlated over time. Together, these findings demonstrate that while returns themselves are largely uncorrelated, their variability displays strong dependence, which is a defining characteristic of volatility clustering in financial markets.

Appendix: Python Code for Question 3 (Volatility Clustering)

Listing 6: Python code for plotting daily BSE Sensex returns

```

1 import matplotlib.pyplot as plt
2
3 # (a) Daily returns plot
4 plt.figure(figsize=(14, 6))
5 plt.plot(returns, color="black", linewidth=0.6)
6 plt.title("Daily Returns of the BSE Sensex (2000--2025)", fontsize=16)
7 plt.xlabel("Time", fontsize=14)
8 plt.ylabel("Returns (%)", fontsize=14)
9 plt.grid(alpha=0.3)
10 plt.tight_layout()
11
12 # Save figure
13 plt.savefig("figures/q3_daily_returns.jpg", dpi=300)
14 plt.close()

```

Listing 7: Python code for plotting the ACF of squared daily BSE Sensex returns

```

1 from statsmodels.graphics.tsaplots import plot_acf
2 import matplotlib.pyplot as plt
3
4 # (b) ACF of squared returns
5 plt.figure(figsize=(10, 5))
6 plot_acf(returns**2, lags=30, zero=False)
7 plt.title("ACF of Squared Daily BSE Sensex Returns", fontsize=14)
8 plt.xlabel("Lag")
9 plt.ylabel("Autocorrelation")
10 plt.tight_layout()
11
12 # Save figure
13 plt.savefig("figures/q3_acf_squared.jpg", dpi=300)
14 plt.close()

```

Question 4: Non-normality of Returns

(a) Skewness and Kurtosis

The distribution of daily BSE Sensex returns exhibits significant departures from normality. The estimated skewness is **(-0.398)**, indicating a moderate negative skewness, which implies that large negative returns occur more frequently than large positive returns. The estimated Pearson kurtosis is **13.048**, which is substantially higher than the normal distribution benchmark of three. This high level of kurtosis indicates pronounced fat tails in the return distribution, reflecting a higher probability of extreme return realizations than predicted under normality.

(b) Comparison with the Normal Distribution

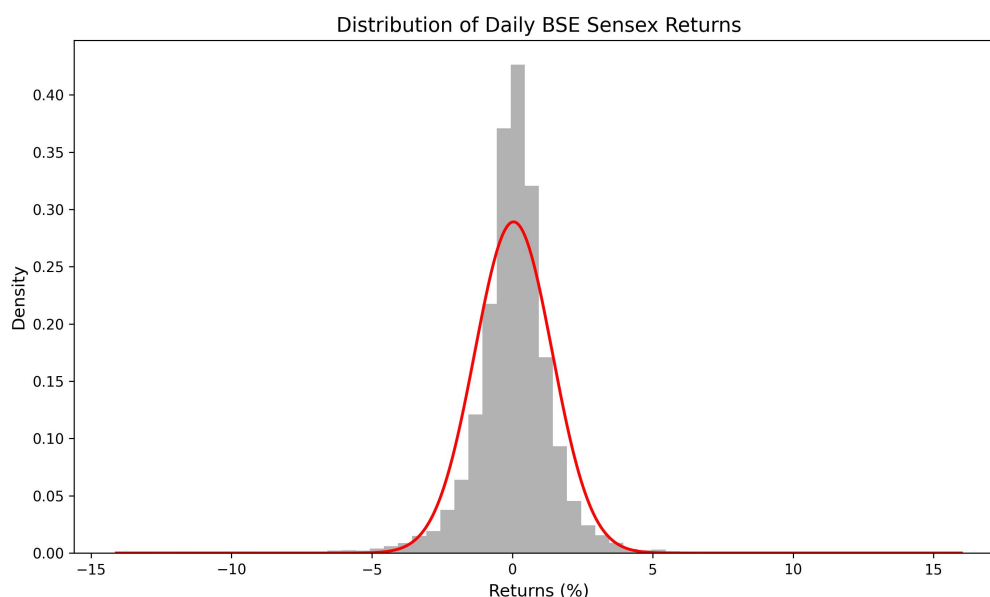


Figure 5: Empirical Distribution of Daily BSE Sensex Returns with Normal Density

Figure 5 compares the empirical distribution of daily BSE Sensex returns with a normal distribution having the same mean and standard deviation. The empirical distribution is more sharply peaked around zero than the normal density, while the normal curve substantially underestimates the probability of extreme returns in both tails. Large positive and negative return realizations occur more frequently in the data than implied by normality. In addition, the left tail appears slightly heavier than the right, indicating mild negative asymmetry. Overall, the figure provides clear visual evidence that the return distribution deviates from the normal distribution.

(c) Implications of Fat Tails for Financial Risk Measurement

The presence of fat tails in the return distribution has important implications for financial risk measurement. As shown by the high kurtosis and the heavy tails in the empirical distribution, extreme return realizations occur more frequently than predicted by the normal distribution. Risk models that assume normality therefore tend to underestimate the probability and magnitude of large losses, leading to downward-biased measures such as Value-at-Risk and Expected Shortfall. Ignoring fat tails can result in insufficient capital buffers and inadequate risk management, particularly during periods of market stress. These findings highlight the need for alternative distributions and volatility models that better capture tail risk in financial markets.

Appendix: Python Code for Question 4 (Non-normality of Returns)

Listing 8: Python code for computing skewness and kurtosis of daily BSE Sensex returns

```
1 from scipy.stats import skew, kurtosis
2 import numpy as np
3
4 # Skewness and kurtosis
5 skewness = skew(returns)
6 kurt = kurtosis(returns, fisher=False) # Pearson kurtosis
7
8 print(f"Skewness           : {skewness.item():.4f}")
9 print(f"Kurtosis (Pearson)   : {kurt.item():.4f}")
```

Listing 9: Python code for plotting the empirical distribution of returns with normal density

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Histogram with normal density overlay
5 plt.figure(figsize=(10, 6))
6
7 # Histogram (density)
8 plt.hist(returns, bins=60, density=True, alpha=0.6, color='gray')
9
10 # Normal density
11 mu = np.mean(returns)
12 sigma = np.std(returns).item()
13 x = np.linspace(returns.min().item(), returns.max().item(), 1000)
14 normal_density = (1/(sigma*np.sqrt(2*np.pi))) * np.exp(-0.5*((x-mu)/sigma)**2)
15
16 plt.plot(x, normal_density, color='red', linewidth=2)
17
18 plt.title("Distribution of Daily BSE Sensex Returns", fontsize=14)
19 plt.xlabel("Returns (%)", fontsize=12)
20 plt.ylabel("Density", fontsize=12)
21 plt.tight_layout()
22
23 # Save figure
24 plt.savefig("figures/q4_distribution.jpg", dpi=300)
25 plt.close()

```

Question 5: Leverage Effect and Interpretation

(a) Negative Returns and Future Volatility

To examine the leverage effect, future volatility is proxied by squared next-day returns and compared conditional on the sign of current returns. The average future volatility following negative returns is approximately 2.145, which is higher than the corresponding value of 1.697 following positive returns. This indicates that negative returns are associated with a stronger increase in subsequent volatility than positive returns, providing empirical evidence of the leverage effect in daily BSE Sensex returns.

(b) Intuitive Explanation of the Leverage Effect

The leverage effect refers to the asymmetric impact of negative and positive returns on future volatility. When stock prices fall, the market value of a firm's equity declines relative to its outstanding debt, effectively increasing financial leverage. Higher leverage makes equity riskier, which leads to an increase in return volatility. As a result, negative returns tend to be followed by larger increases in future volatility than positive returns of similar magnitude. The empirical results in part (a), which show higher future volatility following negative returns, are consistent with this mechanism.

(c) Implications for Volatility Modelling and Risk Management

The presence of the leverage effect has important implications for volatility modelling and risk management. Since negative returns generate larger increases in volatility than positive returns, symmetric volatility models that treat all shocks equally may underestimate risk following market downturns. This motivates the use of asymmetric volatility models, such as EGARCH or GJR-GARCH, which allow negative shocks to have a stronger effect on volatility. Accurately capturing this asymmetry is crucial for reliable risk measurement, stress testing, and capital allocation, particularly during periods of market stress.

Appendix: Python Code for Question 5 (Leverage Effect)

Listing 10: Python code for examining the leverage effect using future volatility

```

1 import numpy as np
2 import pandas as pd
3
4 # Create lagged returns and future volatility proxy
5 df = pd.DataFrame({
6     "returns": returns.squeeze(),
7     "future_vol": returns.squeeze().shift(-1)**2    # squared next-day returns
8 }).dropna()
9
10 # Split based on sign of current return
11 neg_returns = df[df["returns"] < 0]["future_vol"]
12 pos_returns = df[df["returns"] >= 0]["future_vol"]
13
14 # Compare average future volatility
15 neg_vol_mean = neg_returns.mean()
16 pos_vol_mean = pos_returns.mean()
17
18 neg_vol_mean, pos_vol_mean

```